



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных
технологий

Отчет по Модулю 5 – Основы языка Котлин

по дисциплине «Проектирования и разработка мобильных приложений на
языке Котлин»

Выполнили:

Студенты группы ИКБО-65-23

Олефилов Г.Г.

Проверил:

Дворникова Е.М.

2026 г.

Задание 1: Дневник с записью данных в файл

Функционал приложения:

Пользователь ведёт личный дневник в приложении. Каждая запись сохраняется в отдельном файле внутри приватной папки приложения и недоступна извне. Список записей должен обновляться быстро и без полного пересканирования папки при каждой операции.

- Поле ввода для заголовка (опционально) и большое поле для текста заметки;
- Кнопка «Сохранить запись»;
- Список всех сохранённых записей в LazyColumn (с датой и первыми 30–40 символами текста);
- При клике на запись — открывается экран чтения/редактирования;
- Возможность удаления записи (через контекстное меню или кнопку);
- При запуске приложения список загружается автоматически.

Требования к хранению

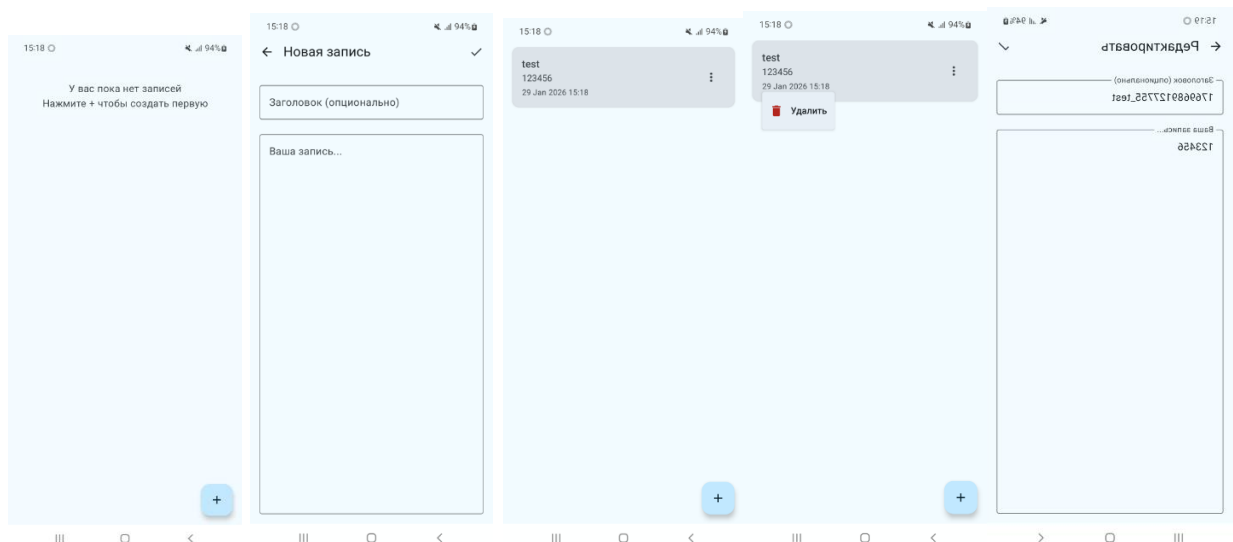
- Каждая запись сохраняется в отдельный файл в `context.filesDir`
- Имя файла — `timestamp_[опционально_заголовок].txt`

Не перегружать весь список целиком при добавлении и удалении записи!

- При сохранении новой записи — добавлять её вручную в начало списка (без повторного сканирования всей папки);
- При удалении записи — удалять её из текущего списка по имени файла (без пересканирования);
- Полное сканирование папки и загрузка всех записей делается только один раз — при запуске приложения (в `init ViewModel`).

Требования к UI

- Главный экран: список записей (LazyColumn) + FAB-кнопка «+ Новая запись»;
- Экран записи: заголовок (опционально), многострочный TextField, кнопки «Сохранить» и «Назад»;
- При долгом нажатии на запись — контекстное меню с пунктом «Удалить»;
- Пустой экран: текст «У вас пока нет записей» + подсказка «Нажмите +, чтобы создать первую».



Моя реализация:

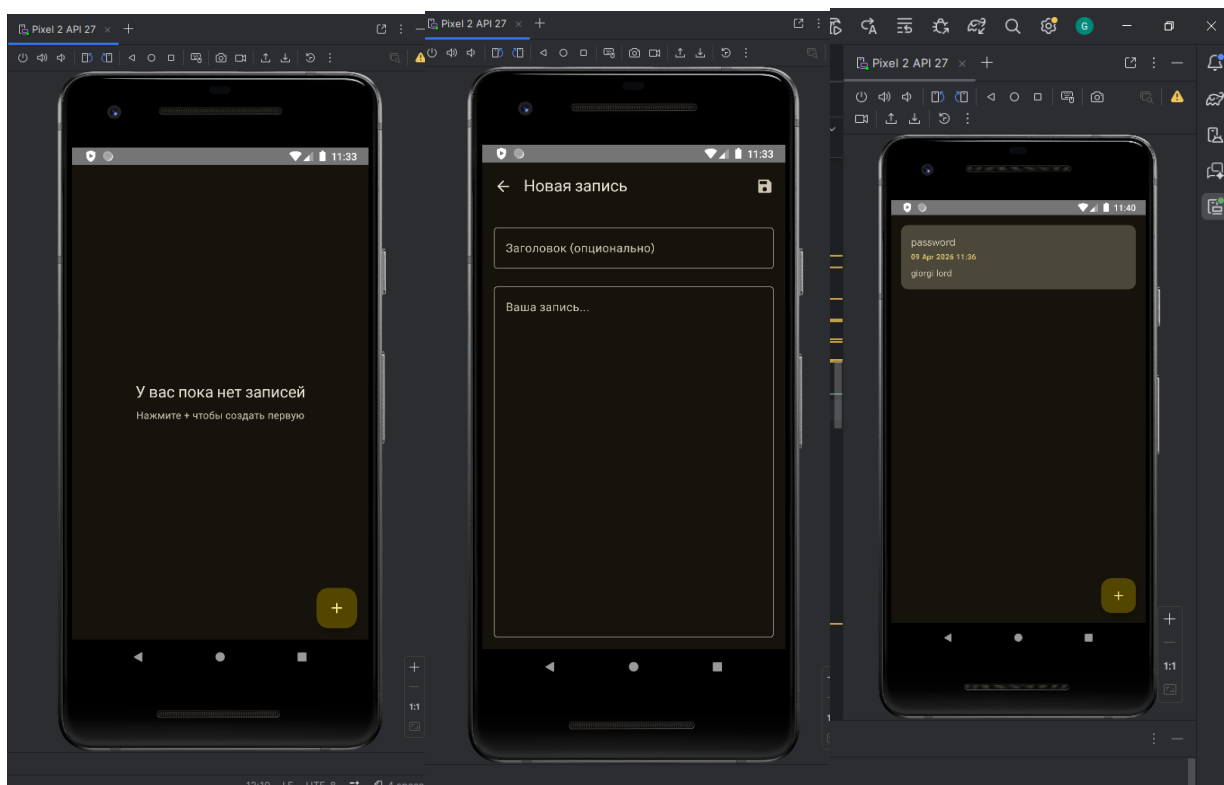


Рисунок 1 – UI создания новой записи

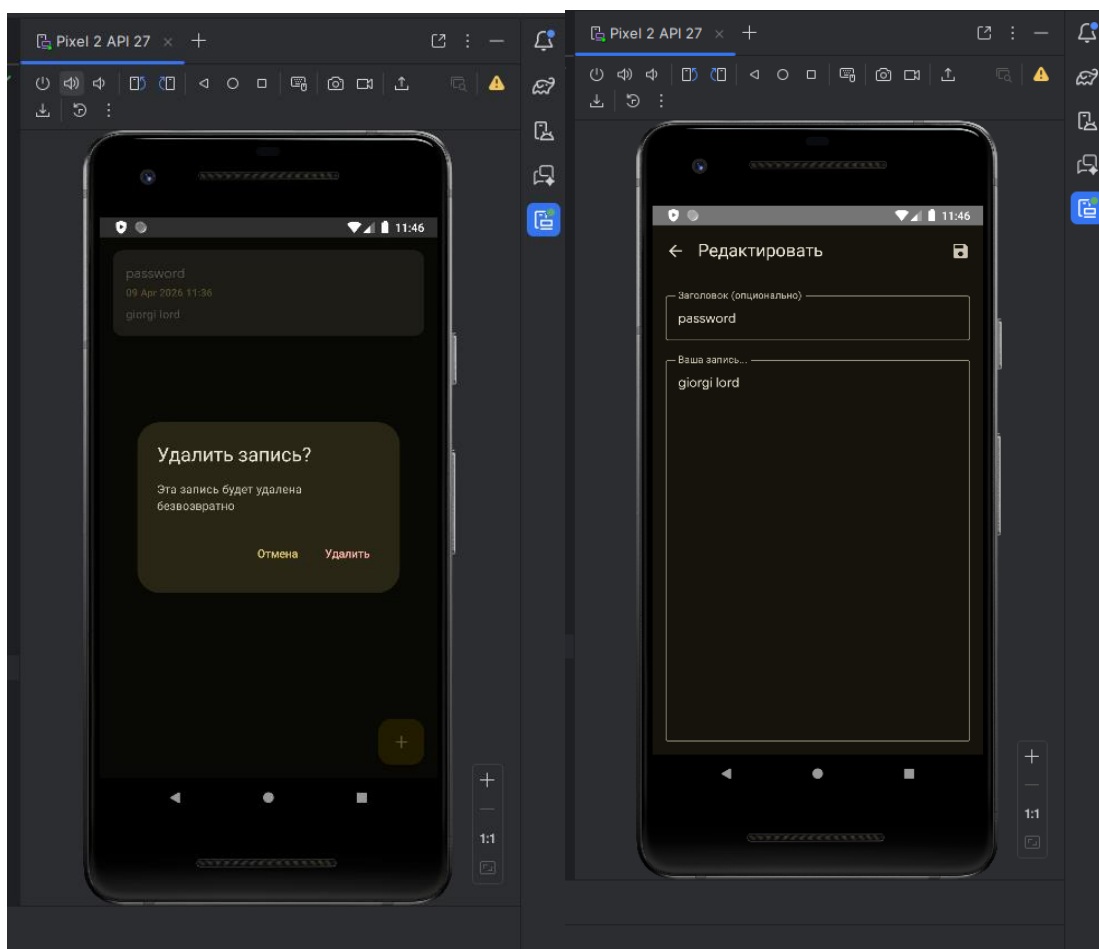


Рисунок 2 – UI редактирования при нажатии на запись или окно с возможным удалением при удержании

Ссылка на гит:

<https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/5module/app/code1>

Задание 2: Галерея фото с кнопкой “Сделать фото”

Функционал приложения:

Необходимо создать простую личная фотогалерею, куда пользователь может добавлять свои фото. Фото сохраняются в приватной папке приложения, но при желании пользователь может экспортировать их в общую галерею.

Функционал, который нужно реализовать:

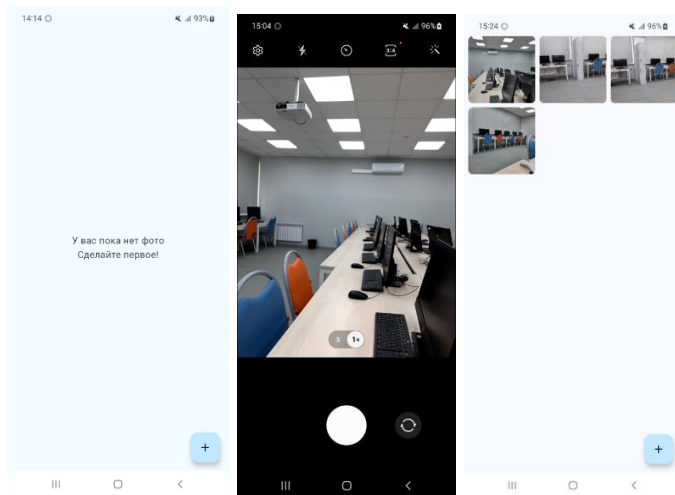
- Кнопка «Сделать фото» - открывает камеру - сохраняет фото в `getExternalFilesDir(Environment.DIRECTORY_PICTURES)`;
- Список всех сделанных фото в `LazyVerticalGrid`;
- Приложение запрашивает `CAMERA` и `WRITE_EXTERNAL_STORAGE` (если нужно).

Требования к файлам:

- Имя файла — `IMG_${SimpleDateFormat("yyyyMMdd_HH:mm:ss")}.jpg`;
- Список фото — сканирование папки при запуске и при добавлении нового фото;

Требования к UI:

- Главный экран — сетка фото (`LazyVerticalGrid`, 3 колонки);
- Пустой экран — текст «У вас пока нет фото» + кнопка «Сделать первое фото»;
- После экспорта — показывать `Snackbar` «Фото добавлено в галерею».



Моя реализация:

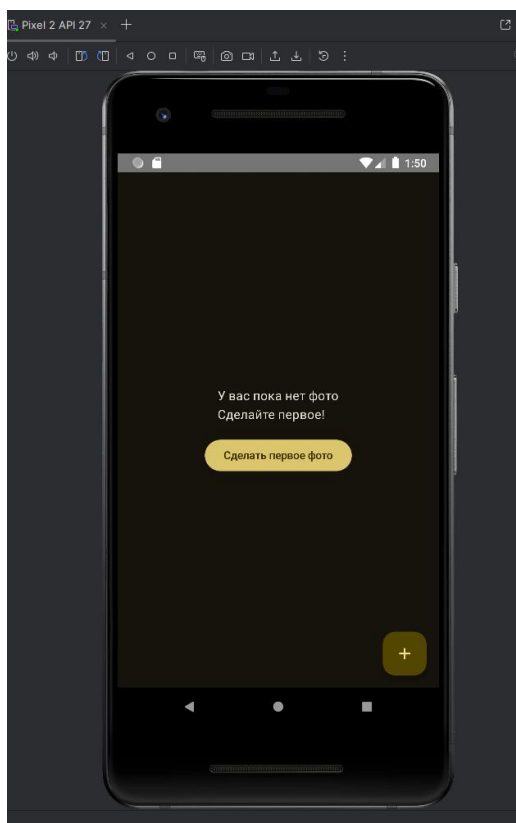


Рисунок 3 - Пустой экран

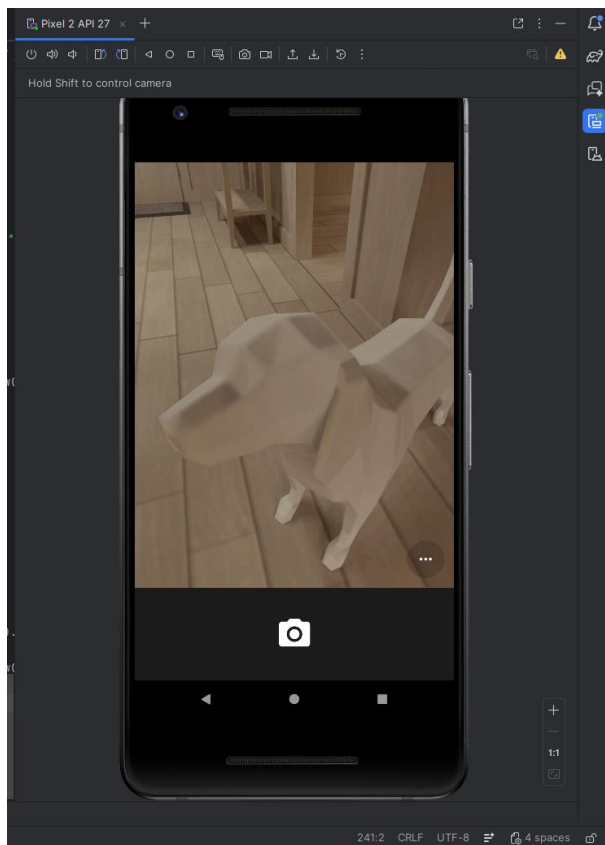


Рисунок 4 - Камера

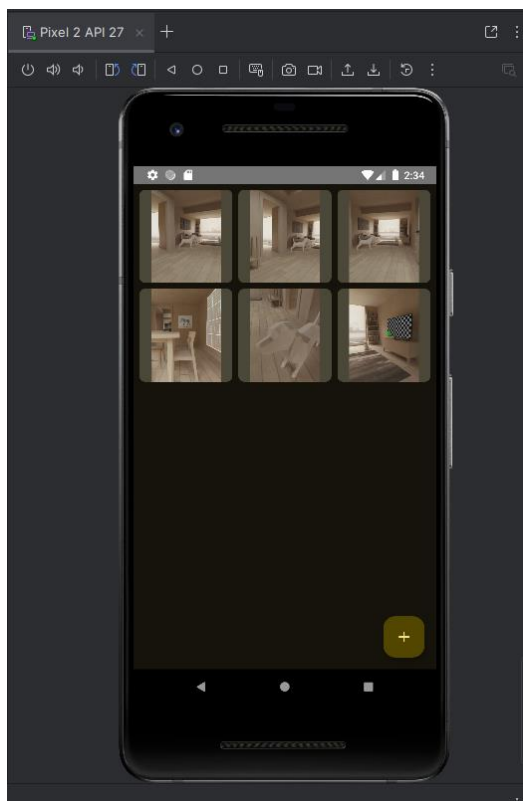


Рисунок 5 - Главный экран

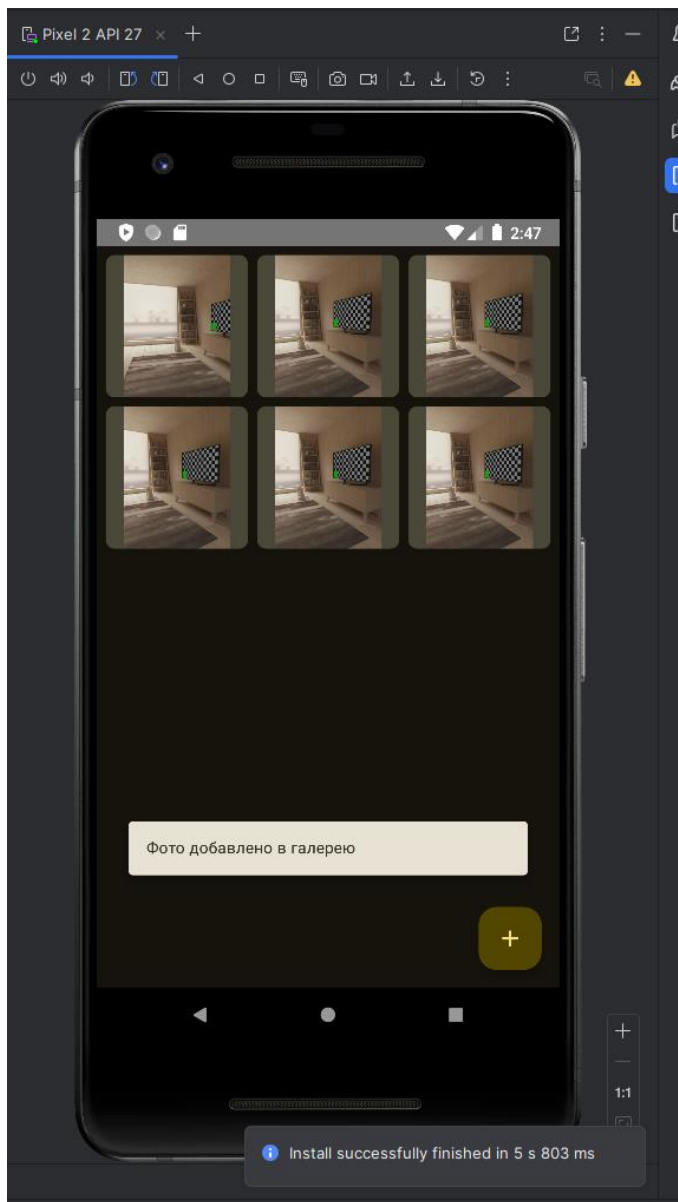


Рисунок 6 - Snackbar «Фото добавлено в галерею».

Ссылка на гит:

<https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/5module/app/code2>

Задание 3: Галерея фото с кнопкой “Сделать фото” (с экспортом)

Функционал приложения:

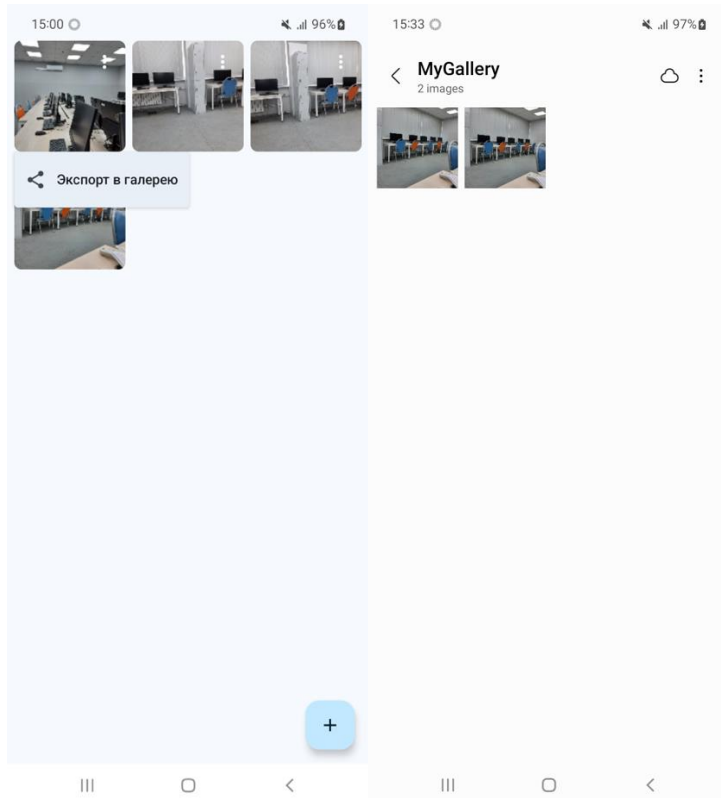
Доработать задание 2 из Модуля 5. Необходимо реализовать экспорт фото в общую галерею.

Функционал, который нужно добавить:

- Кнопка «Экспорт в галерею» на экране просмотра - сохраняет копию в MediaStore (чтобы фото появилось в стандартной галерее телефона)

Требования к UI:

- Главный экран — сетка фото (LazyVerticalGrid, 3 колонки);
- Пустой экран — текст «У вас пока нет фото» + кнопка «Сделать первое фото»;
- Добавить DropDownMenu и пункт «Экспорт в галерею» у каждого изображения в LazyVerticalGrid.



Моя реализация:

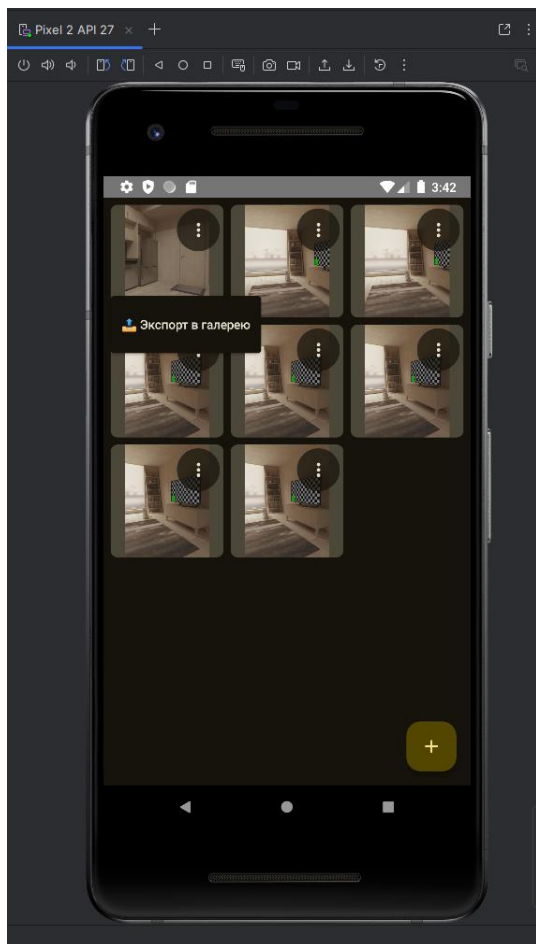


Рисунок 6 - Добавленный «Экспорт в галерею» на главный экран

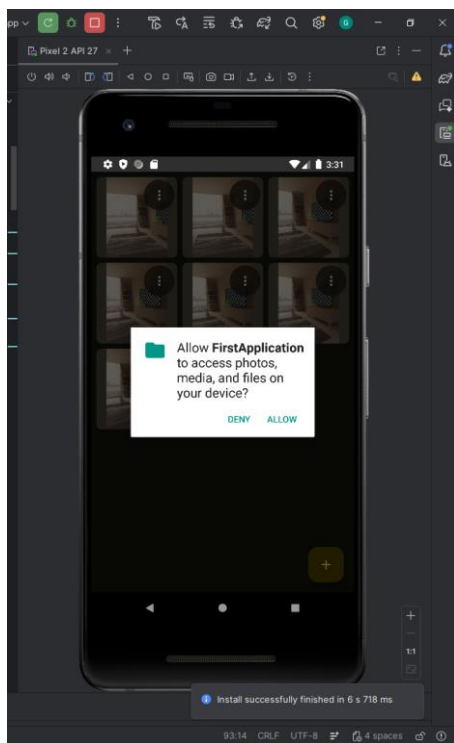


Рисунок 7 – Получение разрешения

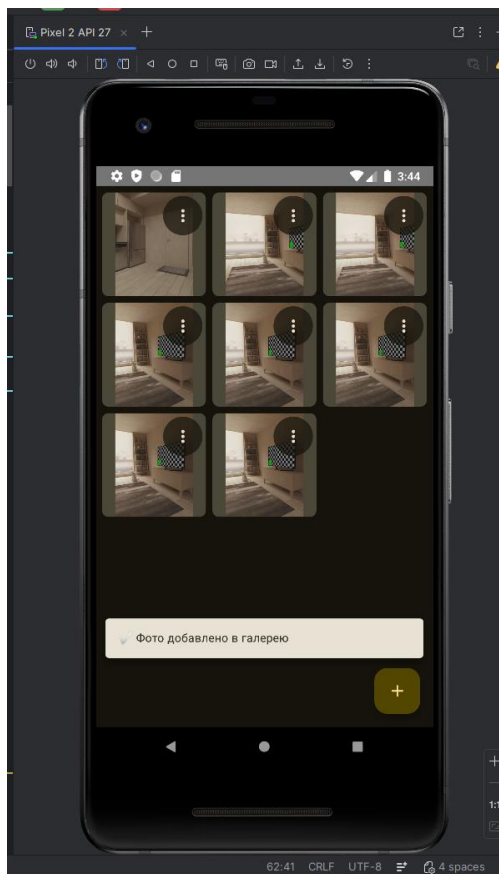


Рисунок 8 - Добавленный DropDownMenu

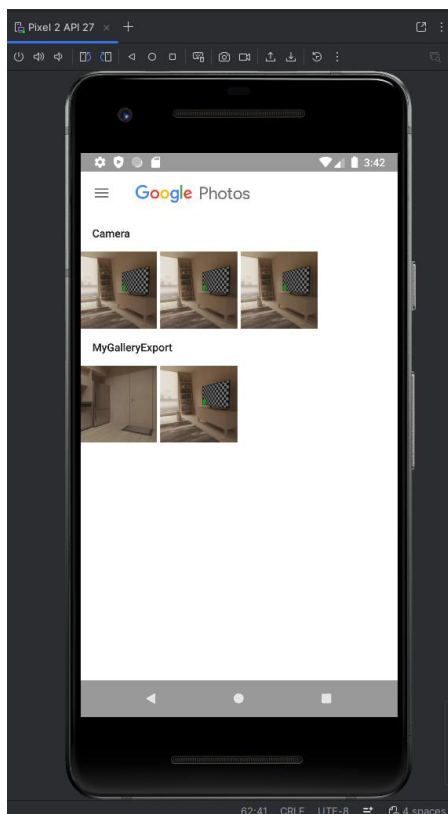


Рисунок 9 – Галерея

Ссылка на гит:

<https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/5module/app/code3>

Задание 4

Доработать приложение TodoList из модуля 3 (задание 7) с использованием:

- Разово импортировать задачи из JSON файла в СУБД SQLite при помощи Room;
- Реализовать функционал добавления, изменения и удаления задачи в СУБД SQLite при помощи Room;
- Добавить возможность изменения цвета фона выполненной задачи, с сохранением значения в Datastore;
- Добавить в TopAppBar управляющий элемент (например switcher):
“Цвет завершенных”.

Структура проекта

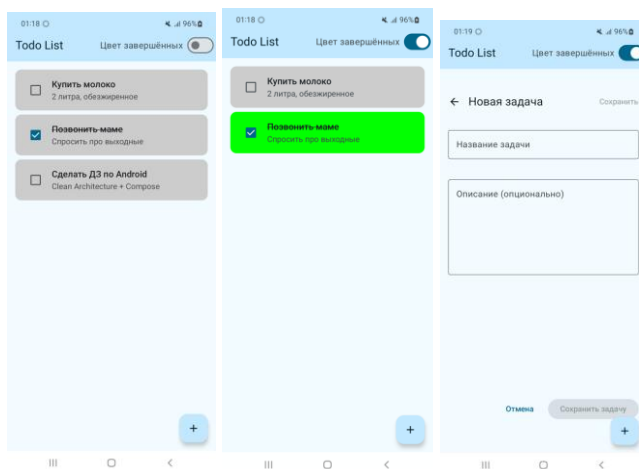
com.example.todolist/

```
├─ data/
|   ├─ local/
|   |   └─ _____
|   └─ repository/
|       └─ _____
|           └─ preferences/
|               └─ _____
|   └─ model/
|       └─ _____
|
├─ domain/
|   └─ model/
|       └─ _____
```

```

| └─ repository/
|   | └─ _____
|   └─ usecase/
|       └─ _____
|       └─ _____
|
| └─ presentation/
|   | └─ ui/
|   |   └─ screen/
|   |     └─ _____
|   |     └─ _____
|   └─ component/
|       └─ _____
|   └─ viewmodel/
|       └─ _____
|
|
|
| └─ navigation/
|   └─ _____
|
|
└─ MainActivity.kt

```



Моя реализация:

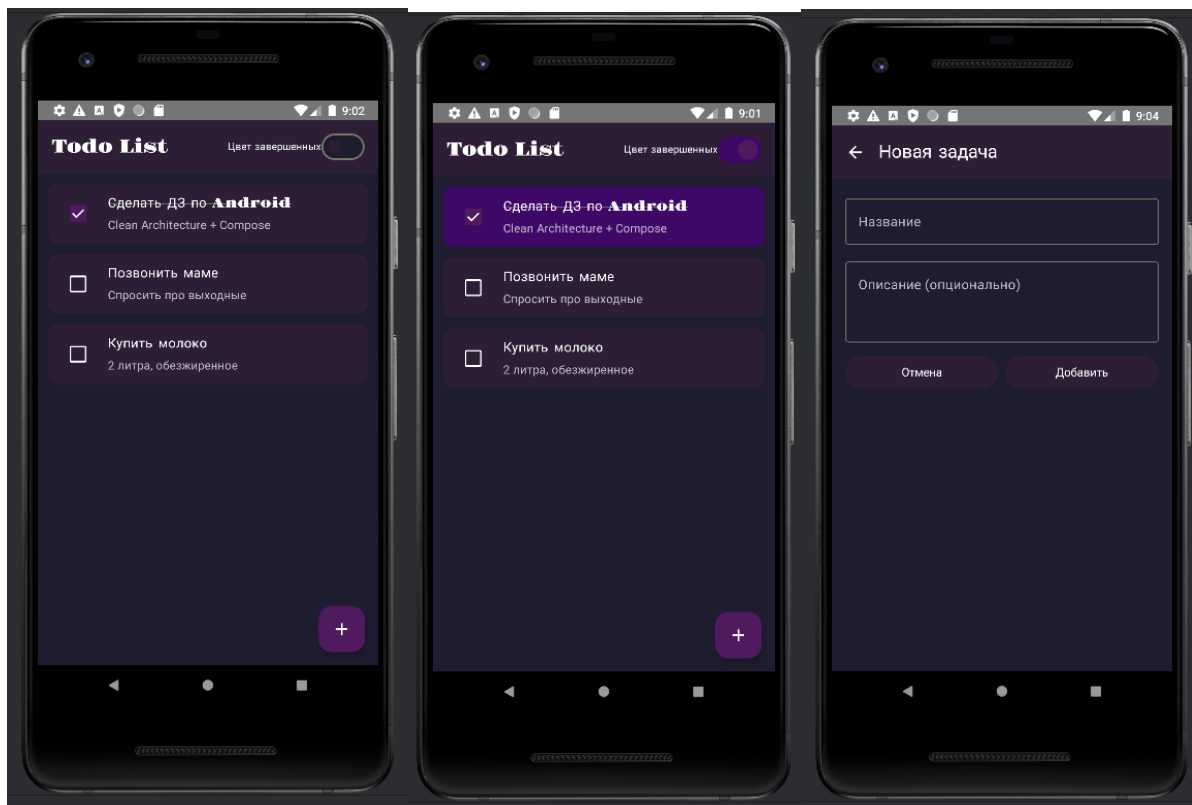


Рисунок 10 – Задание 4

Ссылка на гит:

<https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/5module/app/code4>

Вывод

В ходе выполнения практических работ модуля были систематизированы и углублены знания по управлению данными в операционной системе Android, а также освоены современные подходы к построению устойчивой архитектуры мобильных приложений. В первых заданиях был реализован механизм хранения неструктурированных данных с учетом требований безопасности Scoped Storage, где были изучены различия между внутренним хранилищем для приватных данных и внешним хранилищем для файлов приложения. Особое внимание было уделено оптимизации производительности UI через реализацию обновления списков без полного пересканирования директории, что позволило избежать блокировок главного потока. При разработке фотогалереи был освоен процесс взаимодействия с системной камерой и работа с библиотекой MediaStore для экспорта медиафайлов, что потребовало понимания механизмов запроса разрешений во время выполнения. В задании с приложением ToDoList был осуществлен переход от хранения данных в памяти к использованию реляционной СУБД SQLite посредством библиотеки Room, закрепив принципы Clean Architecture с разделением на слои Data, Domain и Presentation. Использование Flow из библиотеки Kotlin Coroutines позволило реализовать реактивный подход к обновлению интерфейса, а внедрение Jetpack DataStore обеспечило потокобезопасное хранение пользовательских настроек. Применение паттерна MVVM совместно с Jetpack Compose позволило четко разделить ответственность между логикой отображения и бизнес-логикой, правильно управляя жизненным циклом данных и предотвращая потерю состояния при конфигурационных изменениях. Выполнение данных заданий сформировало комплексное понимание выбора оптимального способа хранения в зависимости от типа данных и инструментов для обеспечения надежности и отзывчивости пользовательского интерфейса.